LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 2

STACK & QUEUE

Oleh:

Aufa Thariq Akram **NIM. 2510817310014**

PROGRAM STUDI TEKNOLOGI INFORMASI

FAKULTAS TEKNIK

UNIVERSITAS LAMBUNG MANGKURAT

APRIL 2026

LEMBAR PENGESAHAN

LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA

MODUL 2

Laporan Praktikum Algoritma & Struktur Data Modul 2 : Stack & Queue ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Aufa Thariq Akram
NIM : 2510817310014

Menyetujui,	Mengetahui,
Asisten Praktikum	Dosen Penanggung Jawab Praktikum
Fadhil Syahdama Mahatma Putra	Andreyan Rizky Baskara, S.Kom., M.Kom
NIM. 2410817210026	NIP. 199307032019031011

DAFTAR ISI

LEMBAR PENGESAHAN i
DAFTAR ISI ii
DAFTAR TABEL iii
DAFTAR GAMBAR iv
SOAL 1 1
A. Pembahasan 1
SOAL 2 2
A. Source code 3
B. Output Program 5
C. Pembahasan 6
SOAL 3 9
A. Source Code 10
B. Output Code 12
C. Pembahasan 13
TAUTAN GITHUB 16

DAFTAR TABEL

Tabel 1 Source Code Soal 2 3
Tabel 2 Source Code Soal 3 10

DAFTAR GAMBAR

Gambar 1 Program Screenshot Output Soal 2 Input Data 5
Gambar 2 Program Screenshot Output Soal 2 Hapus Data..... 5
Gambar 3 Program Screenshot Output Soal 2 Tampil Data 6
Gambar 4 Program Screenshot Output Soal 2 Bersihkan Stack 6
Gambar 5 Program Screenshot Output Soal 2 Keluar 6
Gambar 6 Program Screenshot Output Soal 3 Insert 12
Gambar 7 Program Screenshot Output Soal 3 Delete..... 12
Gambar 8 Program Screenshot Output Soal 3 Cetak Queue..... 13
Gambar 9 Program Screenshot Output Soal 3 Quit 13

SOAL 1

Apa perbedaan Stack dengan Queue?

A. Pembahasan

Stack atau tumpukan adalah struktur data yang bekerja dengan prinsip LIFO (Last In, First Out). Dalam konsep ini, data yang paling terakhir dimasukkan ke dalam wadah akan menjadi data yang pertama kali dikeluarkan. Bayangkan saja seperti tumpukan piring di meja; kamu selalu meletakkan piring baru di posisi paling atas dan saat ingin menggunakannya, kamu juga mengambil piring yang berada di posisi paling atas tersebut. Dalam praktikum, proses memasukkan data ini biasanya disebut dengan istilah Push, sedangkan proses mengambilnya disebut Pop. Karena hanya memiliki satu "pintu" akses di bagian atas, data yang pertama kali masuk akan tertumpuk di bawah dan baru bisa diambil setelah semua data di atasnya dikeluarkan.

Sementara itu, Queue atau antrean bekerja dengan logika yang sangat berbeda, yaitu menggunakan prinsip FIFO (First In, First Out). Sesuai dengan namanya, struktur ini meniru cara kerja antrean di dunia nyata, seperti antrean pembeli di kasir atau kendaraan di gerbang tol. Data yang pertama kali masuk ke dalam sistem akan menjadi yang pertama kali diproses atau dikeluarkan. Berbeda dengan Stack, Queue memiliki dua titik akses: satu di bagian belakang untuk memasukkan data baru (disebut Enqueue) dan satu di bagian depan untuk mengeluarkan data (disebut Dequeue). Hal ini memastikan bahwa urutan data tetap terjaga secara kronologis sesuai waktu kedatangannya.

Perbedaan utama di antara keduanya terletak pada manajemen urutannya. Stack lebih cocok digunakan untuk situasi di mana kita perlu membatalkan tindakan terakhir, seperti fitur Undo pada aplikasi pengolah kata atau riwayat penelusuran web. Di sisi lain, Queue sangat ideal untuk sistem yang membutuhkan keadilan dalam pemrosesan, seperti urutan pencetakan dokumen pada printer atau pengaturan lalu lintas data dalam sebuah jaringan, di mana tugas yang datang lebih awal harus diselesaikan terlebih dahulu sebelum melayani tugas yang baru datang.

SOAL 2

Cobalah prog*r*am berikut, running dan analisis hasilnya. Buatlah algoritma untuk program tersebut.

```
int penuh()
{
    if(Tumpuk.atas == max-1)
        return 1;
    else
        return 0;
}

void input (int data)
{
    if (kosong()==1)
    {
        Tumpuk.atas++;
        Tumpuk.data[Tumpuk.atas] = data;
        cout << "Data " << Tumpuk.data[Tumpuk.atas]
             << " Masuk Ke Stack ";
    }
    else if(penuh()==0)
    {
        Tumpuk.atas++;
        Tumpuk.data[Tumpuk.atas] = data;
        cout << "Data "<< Tumpuk.data[Tumpuk.atas]
             << " Masuk Ke Stack ";
    }
    else
        cout << "Tumpukan Penuh";
}

void hapus()
{
    if(kosong()== 0)
    {
        cout << "Data Teratas Sudah Terambil";
        Tumpuk.atas--;
    }
    else
    cout <<" Data Kosong";
}
void tampil()
{
    if (kosong()== 0)
    {
        for(int i = Tumpuk.atas; i>=0; i--)
        {
            cout <<"\nTumpukan Ke " << i << " = "
            <<Tumpuk.data[i];
        }
    }
    else
    cout << "Tumpukan Kosong";
}
void bersih ()
{
    Tumpuk.atas = -1;
    cout << "Tumpukan Kosong !";
}
```

A. Source code

Tabel 1 Source Code Soal 2

```
#include <iostream>
#define max 5

using namespace std;

struct Tumpukan {
    int atas = -1;
    int data[max];
} Tumpuk;

int kosong() {
    if (Tumpuk.atas == -1)
        return 1;
    else
        return 0;
}

int penuh() {
    if (Tumpuk.atas == max - 1)
        return 1;
    else
        return 0;
}

void input(int data_baru) {
    if (penuh() == 0) {
        Tumpuk.atas++;
        Tumpuk.data[Tumpuk.atas] = data_baru;
        cout << "Data " << data_baru << " berhasil
dimasukkan ke Stack." << endl;
    } else {
        cout << "Tumpukan Penuh! Tidak dapat
menambah data." << endl;
    }
}

void hapus() {
    if (kosong() == 0) {
        cout << "Data " << Tumpuk.data[Tumpuk.atas]
<< " telah dihapus dari tumpukan." << endl;
        Tumpuk.atas--;
    } else {
        cout << "Data Kosong!" << endl;
```

```
    }
}

void tampil() {
    if (kosong() == 0) {
        cout << "Isi Tumpukan saat ini:" << endl;
        for (int i = Tumpuk.atas; i >= 0; i--) {
            cout << "Tumpukan Ke " << i << " = " <<
Tumpuk.data[i] << endl;
        }
    } else {
        cout << "Tumpukan Kosong!" << endl;
    }
}

void bersih() {
    Tumpuk.atas = -1;
    cout << "Tumpukan telah dibersihkan!" << endl;
}

int main() {
    int pilihan, nilai;

    do {
        cout << "\n=== MENU STACK ===" << endl;
        cout << "1. Input Data (Push)" << endl;
        cout << "2. Hapus Data (Pop)" << endl;
        cout << "3. Tampil Data" << endl;
        cout << "4. Bersihkan Stack" << endl;
        cout << "5. Keluar" << endl;
        cout << "Pilih menu (1-5): ";
        cin >> pilihan;

        switch (pilihan) {
            case 1:
                cout << "Masukkan angka: ";
                cin >> nilai;
                input(nilai);
                break;
            case 2:
                hapus();
                break;
            case 3:
                tampil();
                break;
```

```
            case 4:
                bersih();
                break;
            case 5:
                cout << "Program selesai." << endl;
                break;
            default:
                cout << "Pilihan tidak valid!" <<
endl;
        }
    } while (pilihan != 5);

    return 0;
}
```

B. Output Program

```
=== MENU STACK ===
1. Input Data (Push)
2. Hapus Data (Pop)
3. Tampil Data
4. Bersihkan Stack
5. Keluar
Pilih menu (1-5): 1
Masukkan angka: 20
Data 20 berhasil dimasukkan ke Stack.
```

Gambar 1 Program Screenshot Output Soal 2 Input Data

```
=== MENU STACK ===
1. Input Data (Push)
2. Hapus Data (Pop)
3. Tampil Data
4. Bersihkan Stack
5. Keluar
Pilih menu (1-5): 2
Data 20 telah dihapus dari tumpukan.
```

Gambar 2 Program Screenshot Output Soal 2 Hapus Data

```
=== MENU STACK ===
1. Input Data (Push)
2. Hapus Data (Pop)
3. Tampil Data
4. Bersihkan Stack
5. Keluar
Pilih menu (1-5): 3
Isi Tumpukan saat ini:
Tumpukan Ke 0 = 20
```

Gambar 3 Program Screenshot Output Soal 2 Tampil Data

```
=== MENU STACK ===
1. Input Data (Push)
2. Hapus Data (Pop)
3. Tampil Data
4. Bersihkan Stack
5. Keluar
Pilih menu (1-5): 4
Tumpukan telah dibersihkan!
```

Gambar 4 Program Screenshot Output Soal 2 Bersihkan Stack

```
=== MENU STACK ===
1. Input Data (Push)
2. Hapus Data (Pop)
3. Tampil Data
4. Bersihkan Stack
5. Keluar
Pilih menu (1-5): 5
Program selesai.
MacBook-Air-AUFA:Modul 2 ASD aufathariqakram$
```

Gambar 5 Program Screenshot Output Soal 2 Keluar

C. Pembahasan

Hasil eksekusi program menunjukkan bahwa struktur data **Stack** berhasil mengimplementasikan prinsip *Last In, First Out*(LIFO) secara konsisten. Ketika data dimasukkan melalui fungsi `input()`, posisi data terbaru selalu berada di urutan teratas

yang ditandai oleh indeks variabel `Tumpuk.atas`. Hal ini terlihat jelas saat fungsi `tampil()` dijalankan, di mana urutan data yang muncul dimulai dari angka yang terakhir kali diinputkan hingga ke elemen pertama. Begitu pula saat dilakukan penghapusan menggunakan fungsi `hapus()`, program secara otomatis mengambil elemen yang berada di posisi paling atas dengan cara menurunkan nilai indeks `atas`, sehingga data yang lebih dulu masuk tetap aman di posisi bawah. Mekanisme ini membuktikan bahwa akses pada tumpukan bersifat terbatas hanya pada satu ujung saja, sehingga sangat efektif untuk menangani operasi yang membutuhkan urutan balik atau pembatalan tindakan terakhir

Pembahasan Syntax Code

- #include <iostream>: Baris ini adalah perintah untuk menyiapkan alat tulis dan kertas digital agar program bisa menampilkan teks ke layar dan membaca apa yang kita ketik.
- using namespace std: Ini digunakan agar kita bisa menulis perintah seperti cout atau cin secara langsung tanpa perlu menambahkan kode tambahan di depannya.
- #define max 5: Di sini kita membuat aturan bahwa lemari penyimpanan kita hanya memiliki 5 laci saja.
- struct Stack: Kita sedang membuat sebuah kotak besar bernama Stack yang akan menampung laci-laci data dan sebuah penunjuk posisi.
- int data[max]: Ini adalah laci-laci tempat kita menyimpan angka-angka sesuai batas maksimal yang sudah kita buat tadi.
- int atas: Variabel ini bertugas sebagai jari yang menunjuk ke arah laci mana yang paling atas saat ini.
- Stack Tumpuk: Kita memberi nama kotak penyimpanan tersebut dengan sebutan Tumpuk.
- int kosong(): Sebuah cara untuk mengecek apakah kotak penyimpanan kita masih kosong atau sudah ada isinya.
- if (Tumpuk.atas == -1): Jika jari penunjuk berada di posisi -1, itu artinya kotak benar-benar kosong karena laci pertama dimulai dari angka 0.

- int penuh(): Cara untuk memastikan apakah semua laci sudah terisi penuh atau masih ada sisa ruang.
- if (Tumpuk.atas == max - 1): Jika jari penunjuk sudah sampai di laci nomor 4 (dari total 5 laci), berarti kotak sudah tidak bisa diisi lagi.
- void input: Ini adalah instruksi untuk memasukkan data baru ke dalam tumpukan.
- Tumpuk.atas++: Sebelum menaruh barang, kita menaikkan jari penunjuk satu tingkat ke atas untuk menyiapkan tempat baru.
- void hapus: Instruksi untuk mengambil barang yang berada di laci paling atas.
- Tumpuk.atas--: Kita cukup menurunkan jari penunjuk satu tingkat ke bawah, sehingga barang yang lama tidak lagi dianggap ada dalam antrean akses.
- void tampil: Ini adalah perintah untuk melihat semua barang yang ada di dalam laci secara berurutan.
- for (int i = Tumpuk.atas; i >= 0; i--): Kita menyuruh program untuk melihat isi laci mulai dari posisi jari penunjuk saat ini sampai ke laci paling bawah.
- void bersih: Perintah instan untuk mengosongkan semua laci sekaligus tanpa harus mengambilnya satu-satu.
- Tumpuk.atas = -1: Kita tinggal mengembalikan jari penunjuk ke posisi -1, sehingga kotak dianggap kosong kembali.
- int main(): Ini adalah tempat dimulainya semua kegiatan atau menu utama dari program yang kita buat.
- do while (pilihan != 5): Program akan terus berjalan dan menampilkan menu pilihan sampai kita menekan tombol nomor 5 untuk berhenti.

SOAL 3

Cobalah contoh program berikut, running dan analisis hasilnya!

```
#include<iostream>
#include<conio.h>
#include<stdlib.h>
#define n 10
using namespace std;

void INSERT();
void DELETE();
void CETAKLAYAR();
void Inisialisasi();
void RESET();
int PIL,F,R;
char PILIHAN [1],HURUF;
char Q[n];
int main ( )
{
     Inisialisasi();
     do
     {
           cout<<"QUEUE"<<endl;
           cout<<"==============="<<endl;
           cout<<"1. INSERT"<<endl;
           cout<<"2. DELETE"<<endl;
           cout<<"3. CETAK QUEUE"<<endl;
           cout<<"4. QUIT"<<endl;
           cout<<"PILIHAN : "; cin>>PILIHAN;
           PIL=atoi(PILIHAN);
   switch (PIL)
   {
   case 1:
         INSERT ();
         break;
   case 2:
         DELETE();
         break;
   case 3:
         CETAKLAYAR ();
         break;
   default:
         cout<<"TERIMA KASIH"<<endl;
         break;
   }
   cout<<"press any key to continue"<<endl;
   getch();
   system("cls");
}
while (PIL<4);
}
```

A. Source Code

Tabel 2 Source Code Soal 3

```
#include <iostream>
#include <stdlib.h>

#define n 10
using namespace std;

void INSERT();
void DELETE();
void CETAKLAYAR();
void Inisialisasi();
void RESET();

int PIL, F, R;
char HURUF;
char Q[n];

void Inisialisasi() {
    F = R = -1;
}

void RESET() {
    F = R = -1;
    cout << "Queue dikosongkan" << endl;
}

void INSERT() {
    if (R == n - 1) {
        cout << "Queue Penuh!" << endl;
    } else {
        cout << "Masukkan data: ";
        cin >> HURUF;
        if (F == -1) {
            F = R = 0;
        } else {
            R++;
        }
        Q[R] = HURUF;
        cout << "Data masuk ke queue" << endl;
    }
}

void DELETE() {
    if (F == -1) {
```

```
        cout << "Queue Kosong!" << endl;
    } else {
        cout << "Data keluar: " << Q[F] << endl;
        if (F == R) {
            F = R = -1;
        } else {
            F++;
        }
    }
}

void CETAKLAYAR() {
    if (F == -1) {
        cout << "Queue kosong" << endl;
    } else {
        cout << "Isi Queue: ";
        for (int i = F; i <= R; i++) {
            cout << Q[i] << " ";
        }
        cout << endl;
    }
}

int main() {
    Inisialisasi();
    do {
        cout << "QUEUE" << endl;
        cout << "===============" << endl;
        cout << "1. INSERT" << endl;
        cout << "2. DELETE" << endl;
        cout << "3. CETAK QUEUE" << endl;
        cout << "4. QUIT" << endl;
        cout << "PILIHAN : "; cin >> PIL;

        switch (PIL) {
            case 1:
                INSERT();
                break;
            case 2:
                DELETE();
                break;
            case 3:
                CETAKLAYAR();
                break;
            default:
```

```
                    cout << "TERIMA KASIH" << endl;
                    break;
            }

            if (PIL < 4) {
                cout << "press any key to continue" << endl;
                cin.ignore();
                system("clear");
            }

        } while (PIL < 4);

        return 0;
}
```

B. Output Code

```
===============
1. INSERT
2. DELETE
3. CETAK QUEUE
4. QUIT
PILIHAN : 1
Masukkan data: 20
Data masuk ke queue
press any key to continue
QUEUE
```

Gambar 6 Program Screenshot Output Soal 3 Insert

```
1. INSERT
2. DELETE
3. CETAK QUEUE
4. QUIT
PILIHAN : 2
Data keluar: 1
press any key to continue
QUEUE
```

Gambar 7 Program Screenshot Output Soal 3 Delete

```
1. INSERT
2. DELETE
3. CETAK QUEUE
4. QUIT
PILIHAN : 3
Queue kosong
press any key to continue
QUEUE
---------------
```

Gambar 8 Program Screenshot Output Soal 3 Cetak Queue

```
1. INSERT
2. DELETE
3. CETAK QUEUE
4. QUIT
PILIHAN : 4
TERIMA KASIH
MacBook-Air-AUFA:Modul 2 ASD aufathariqakram$
```

Gambar 9 Program Screenshot Output Soal 3 Quit

C. Pembahasan

Analisis hasil eksekusi program ini menunjukkan bahwa struktur data Queue berhasil dijalankan dengan menggunakan prinsip First In First Out (FIFO), di mana data yang masuk paling awal akan menjadi yang pertama kali keluar. Melalui fungsi INSERT, setiap karakter baru ditambahkan ke posisi belakang antrean dengan menggeser penunjuk Rear ke depan, sementara posisi depan tetap dikunci oleh penunjuk Front. Saat proses DELETE dilakukan, program secara otomatis mengambil data dari posisi Front, yang membuktikan bahwa urutan pengambilan data sangat bergantung pada waktu kedatangannya, bukan pada posisi terakhir seperti pada Stack. Program juga dilengkapi dengan sistem pengecekan kondisi antrean; jika penunjuk Rear sudah mencapai batas maksimal (n-1), maka program akan mendeteksi kondisi penuh dan menolak input data baru. Sebaliknya, jika Front bernilai -1, program akan menginformasikan bahwa antrean dalam keadaan kosong. Keseluruhan

mekanisme ini memastikan pengelolaan data karakter dalam array berjalan secara teratur dan adil sesuai urutan antrean.

Pembahasan Syntax Code

- #include <iostream>: Baris ini berfungsi untuk memanggil perpustakaan standar agar program bisa menjalankan perintah input dan output dasar.
- #include <conio.h>: Digunakan untuk mendukung fungsi getch agar layar tidak langsung tertutup setelah proses selesai.
- #include <stdlib.h>: Perpustakaan ini dipanggil untuk menggunakan fungsi atoi dan system cls untuk membersihkan layar.
- #define n 10: Baris ini menetapkan bahwa kapasitas maksimal antrean adalah 10 data karakter.
- using namespace std;: Digunakan agar penulisan perintah standar seperti cout tidak perlu diawali dengan std:: secara berulang.
- int F, R;: Variabel F (Front) menandakan posisi depan antrean, sedangkan R (Rear) menandakan posisi belakang antrean.
- char Q[n];: Ini adalah tempat penyimpanan atau array yang digunakan untuk menampung data antrean dalam bentuk karakter.
- void Inisialisasi(): Fungsi ini dijalankan di awal untuk memastikan antrean dalam keadaan bersih dengan mengatur posisi F dan R ke angka -1.
- void INSERT(): Instruksi untuk menambahkan data baru ke dalam antrean dari posisi belakang.
- if (R == n - 1): Program memeriksa apakah posisi belakang sudah mencapai batas maksimal 10 data atau belum.
- R++: Jika antrean masih ada ruang, posisi belakang akan digeser maju satu langkah sebelum data baru dimasukkan.
- Q[R] = HURUF;: Data karakter yang diketik pengguna disimpan ke dalam laci antrean pada posisi belakang.

- void DELETE(): Fungsi ini digunakan untuk mengeluarkan data dari posisi paling depan sesuai urutan kedatangan.
- if (F == R): Kondisi khusus untuk memeriksa apakah hanya tinggal satu data tersisa, jika iya maka antrean dikosongkan kembali ke posisi -1.
- F++: Jika masih ada data lain, posisi depan akan digeser maju sehingga data yang lama dianggap sudah keluar.
- void CETAKLAYAR(): Instruksi untuk menampilkan semua data yang saat ini sedang mengantre di layar.
- for (int i = F; i <= R; i++): Program melakukan perulangan untuk membaca isi antrean mulai dari urutan depan sampai ke belakang.
- PIL = atoi(PILIHAN);: Berfungsi mengubah masukan menu yang berupa teks menjadi angka agar bisa diproses oleh perintah switch.
- system("cls");: Perintah untuk membersihkan seluruh tulisan di layar terminal agar tampilan menu tetap rapi.
- while (PIL < 4);: Program akan terus mengulang tampilan menu selama pengguna tidak memilih nomor 4 untuk keluar.

TAUTAN GITHUB

https://github.com/IntraGH/Algoritma-dan-Struktur-Data